Analyse de Séquences ADN et Implémentation d'un Cadre de Test Unitaire via JUnit : Approche et Documentation

Résumé

JUnit 5 est le standard de fait pour l'écriture de **tests unitaires** en Java, permettant la création de suites de tests répétables avec des résultats cohérents . Ce framework se compose de **trois modules** : **JUnit Platform**, **JUnit Jupiter** et **JUnit Vintage**. Cette architecture offre une flexibilité pour l'exécution et de la compatibilité. Ce rapport détaille son **architecture**, son **processus d'installation** via Maven et les étapes pour réaliser un **premier test**, ainsi il spécifie un **exemple de démonstration** qui illustre la **démarche de test** suivie.

1 Introduction

JUnit est un cadre de test open source utilisé pour aider la vérification des petites parties individuelles du code (des méthodes ou des fonctions) qui fonctionnent correctement.
L'objectif principal est de s'assurer que le code se comporte comme prévu, même lorsque des changements ou de nouvelles fonctionnalités sont ajoutés.
JUnit s'intègre parfaitement avec les outils de construction populaires comme Maven ou Gradle, ainsi qu'avec les principaux IDE tels qu'IntelliJ IDEA, Eclipse et VS Code.

Figure 1 – JUnit Logo

2 Architecture

L'architecture de JUnit 5 se différe des versions précédentes par sa structure modulaire. Elle est divisée en trois composants principaux :

JUnit Platform, JUnit Jupiter et JUnit Vintage

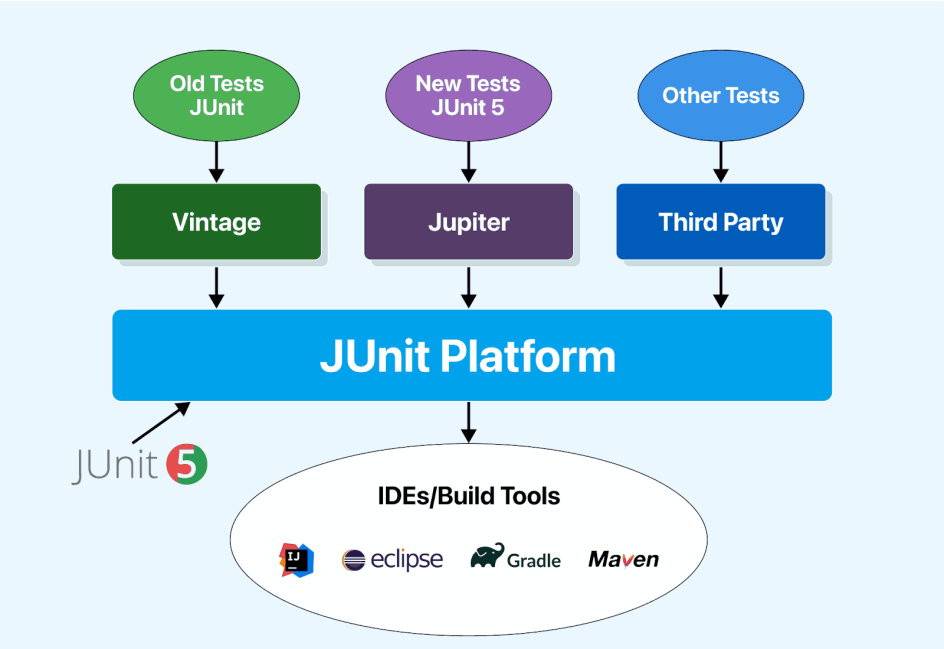

FIGURE 2 – JUnit Architecture

— **JUnit Vintage**
— **JUnit Jupiter**
— **JUnit Platform** : Ce module est **la base** pour lancer les frameworks de test sur la JVM . Il définit l'API **TestEngine** qui permet de développer des frameworks de test s'exécutant sur la plateforme . JUnit Platform assure aussi la **liaison** avec les **outils de build** (Maven, Gradle) et les **IDE** populaires comme IntelliJ IDEA, Eclipse et VS Code

3 Classification

Les frameworks de test unitaire libres sont généralement classifiés selon le langage cible. Ils appartiennent tous à la famille xUnit, un ensemble de frameworks inspirés du SmallTalk SUnit original de Kent Beck.

Framework	Langage cible	Licence	Remarques
JUnit	Java	Eclipse Public License	Référence pour la JVM ; base de la famille xUnit Java
CppUnit	C++	LGPL	Portage de JUnit pour C++
CUnit	C	GNU LGPL	Framework léger pour le langage C pur
PHPUnit	PHP	BSD 3-Clause	Standard de facto pour les tests PHP
HTMLUnit	HTML / JavaScript	Apache 2.0	Simule un navigateur headless pour tester les interfaces web
NUnit	C# / .NET	MIT	Équivalent JUnit pour l'écosystème Microsoft
PyUnit	Python	PSF License	Intégré dans la bibliothèque standard Python (`unittest`)
RSpec	Ruby	MIT	Orienté BDD, très utilisé dans l'écosystème Rails

Table 1 – Classification des frameworks de test libres

Contrairement à HTMLUnit qui cible les tests d'interface, ou à CUnit qui opère au niveau système en C, JUnit se positionne spécifiquement sur les tests unitaires et d'intégration de composants Java, avec un riche écosystème d'extensions (Mockito pour les mocks, AssertJ pour les assertions fluentes, JaCoCo pour la couverture de code).

4 Installation

4.1 Prérequis

- **Java Development Kit** : JDK 8 ou supérieur
- **VS Code IDE**
- **Maven**

4.2 Installation des outils

Installation du JDK 17 :

```
File  Edit  View  Search  Terminal  Help
wala@wala-VivoBook-ASUSLaptop-X515EP-X515EP:~$ sudo apt update
```

Figure 3 – Step 1 : Updating system's local package database

```
wala@wala-VivoBook-ASUSLaptop-X515EP-X515EP:~$ sudo apt install install openjdk-17-jdk
```

Figure 4 – Step 2 : Installing a specific version (JDK 17)

```
wala@wala-VivoBook-ASUSLaptop-X515EP-X515EP:~$ java --version
java 17.0.7 2023-04-18 LTS
Java(TM) SE Runtime Environment (build 17.0.7+8-LTS-224)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.7+8-LTS-224, mixed mode, sharing)
```

Figure 5 – Step 3 : Checking the installation

Installation des extensions nécessaires : Deux extensions indispensables

- Extension Pack for Java

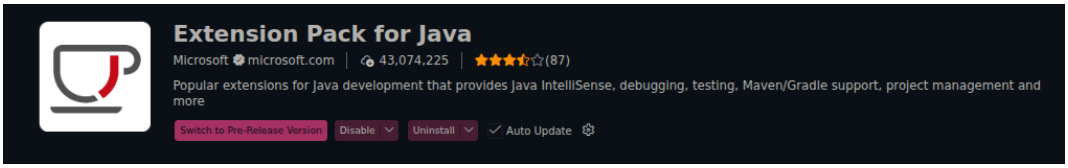

FIGURE 6 – Extension Pack for Java

- Java Test Runner

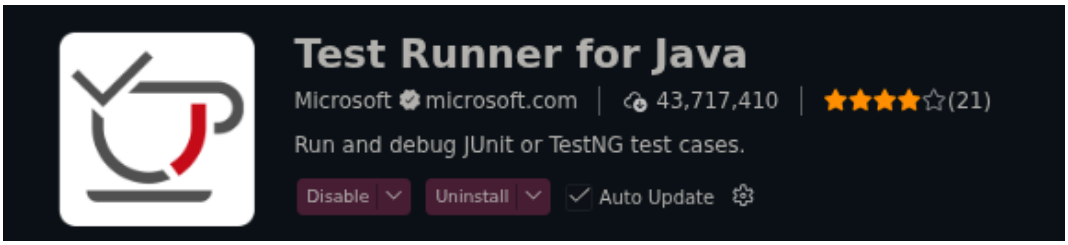

FIGURE 7 – Test Runner for Java

5 Premier lancement

5.1 Création et configuration du projet

Création du projet JAVA

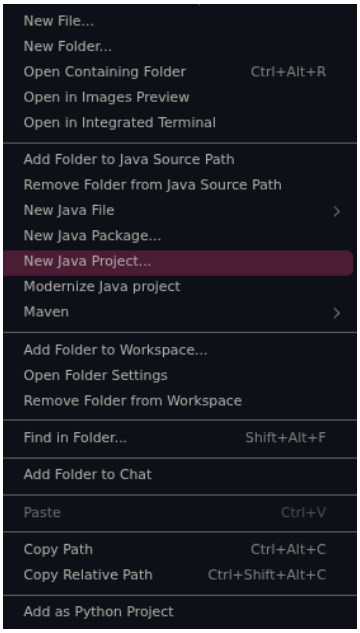

FIGURE 8 – 1- Java Project Creation

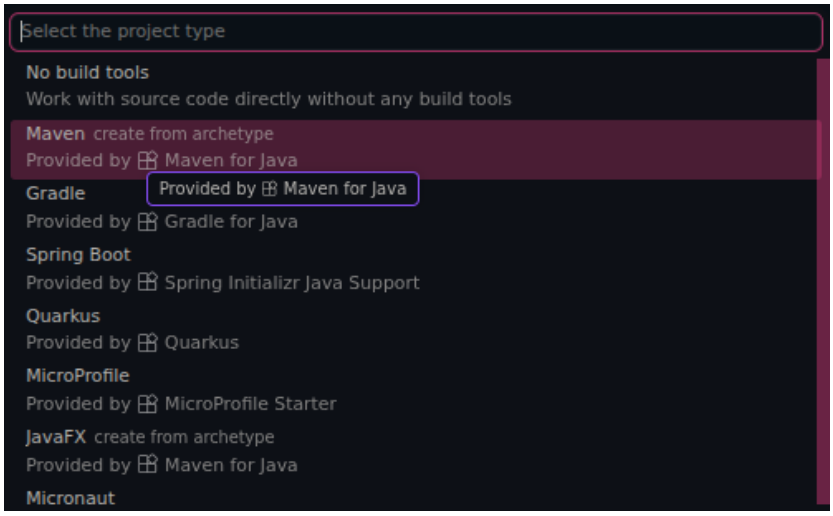

FIGURE 9 – 2- Project Type Specification

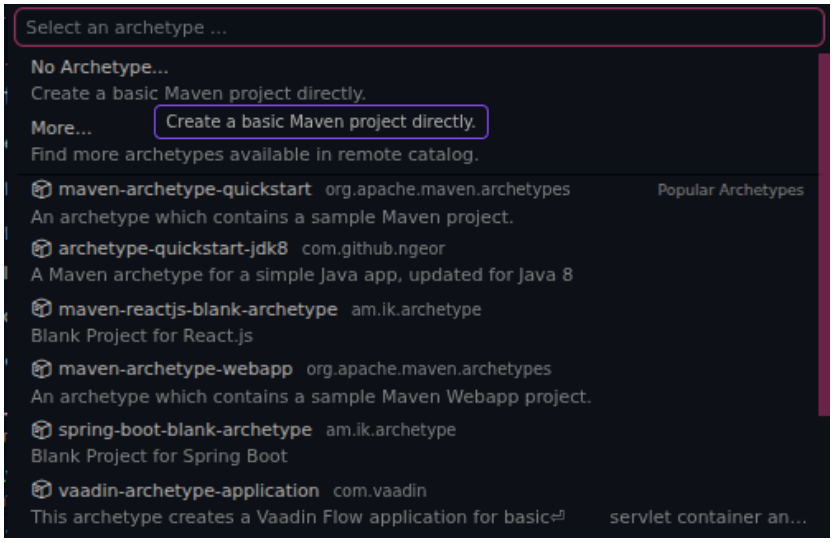

Figure 10 – 3- Archtype Specification

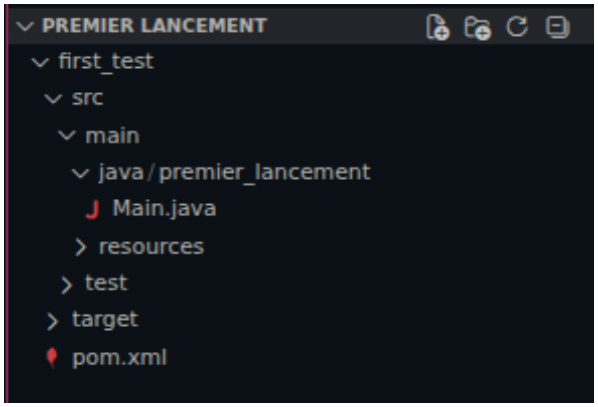

Figure 11 – Project Structure

Ajout de JUnit 5 (avec Maven)

```
<dependency>
<groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
</dependency>
```

Figure 12 – Depandancies in pom.xml

5.2 Création d'un premier test

Code source de l'exemple Calculatrice

```java
package premier_lancement;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

FIGURE 13 – Code Source of Calculator.java

Implémentation des tests

```java
import premier_lancement.*;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
class CalculatorTest {

    @Test
    void testAddition() {
        int result = 2 + 3;
        assertEquals(expected: 5, result);
    }
    @Test
    void testAdditionWithCalculator() {
        Calculator calculator = new Calculator();
        int result = calculator.add(a: 2, b: 3);
        assertEquals(expected: 5, result);
}
    @Test
    void testAdditionWithNegativeNumbers() {
        Calculator calculator = new Calculator();
        int result = calculator.add(a: 2, -3);
        boolean is_negatif = false;
        if (result < 0) {
            is_negatif = true;
        }

        assertTrue(is_negatif);

    }}
```

FIGURE 14 – Test Logic Implementation

Exécution des tests

```
1  import premier_lancement.*;
2  import static org.junit.jupiter.api.Assertions.*;
3  import org.junit.jupiter.api.Test;
   Click to run tests, right click for more options
6      @Test
7      void testAddition() {
8          int result = 2 + 3;
9          assertEquals(expected: 5, result);
10     }
11     @Test
12     void testAdditionWithCalculator() {
13         Calculator calculator = new Calculator();
14         int result = calculator.add(a: 2, b: 3);
15         assertEquals(expected: 5, result);
16 }
```

FIGURE 15 – Test Execution

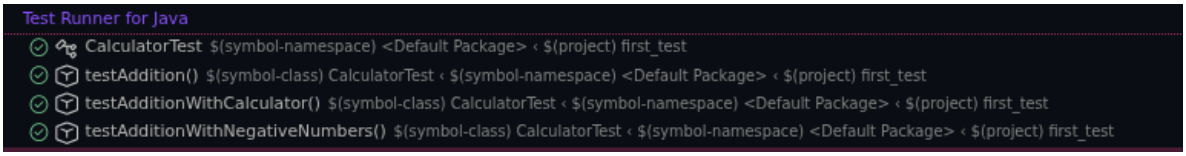

FIGURE 16 – Test Results

Code Source : Lien de github repository

6 Exemple de démonstration

Cette partie présente un exemple de la démarche de test appliquée au programme DNAAnalyzer, un analyseur de séquences ADN développé en Java. Il couvre l'ensemble du cycle de test, depuis la présentation du programme original jusqu'à la clôture du rapport.

6.1 Programme original

6.1.1 Présentation

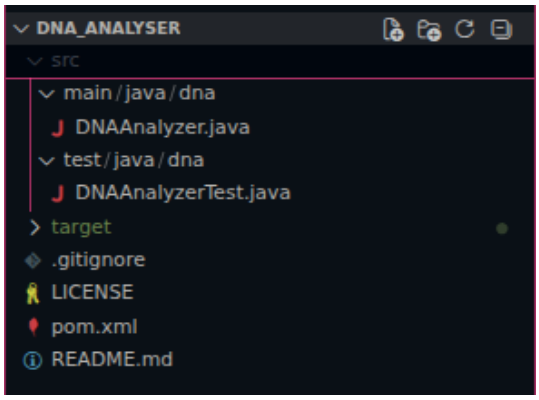

FIGURE 17 – Code Structure

DNAAnalyzer est une classe Java situee dans le package dna. Elle fournit un ensemble de fonctionnalites permettant l'analyse de sequences d'acides desoxyribonucleiques (ADN).

Le programme est concu pour la bioinformatique et propose les operations suivantes :

• **validate(String seq)** : verifie si une chaine de caracteres constitue une sequence ADN valide (lettres A, T, C, G uniquement, insensible a la casse).

• **complement(String seq)** : retourne la partie complementaire inverse selon les regles (A-T, C-G).

• **gcContent(String seq)** : calcule le taux de bases G/C en pourcentage.

• **isPalindrome(String seq)** : detecte si une sequence constitue un palindrome genetique (egal a son propre complementaire inverse).

• **transcribe(String seq)** : transcrit l'ADN en ARN messager en remplacant chaque T par un U.

La structure interne repose sur une **Map<Character, Character>** constante **COMPLEMENT-MAP** qui associe chaque nucleotide a son complementaire. Toutes les methodes publiques effectuent

une validation preventive via **validate()** et levent une **IllegalArgumentException** en cas de sequence invalide ou nulle.

Code Source : Lien de github repository

6.1.2 Spécification des exigences

Les exigences fonctionnelles du programme :

ID	Méthode	Exigence fonctionnelle	Type de retour
EF01	**validate()**	Retourne true si la séquence ne contient que A, T, C, G (insensible à la casse). Retourne false si vide. Lève IllegalArgumentException si null.	boolean
EF02	**complement()**	Retourne le complémentaire inverse en majuscules. Lève IllegalArgumentException si invalide ou null.	String
EF03	**gcContent()**	Retourne le pourcentage de bases G/C avec précision décimale. Lève IllegalArgumentException si invalide, vide ou null.	double
EF04	**isPalindrome()**	Retourne true si la séquence est égale à son complémentaire inverse. Retourne false si null, impaire ou invalide.	boolean
EF05	**transcribe()**	Remplace T par U en majuscules. Lève IllegalArgumentException si invalide ou null.	String

Table 2 – Spécifications des méthodes de traitement de séquences ADN

Les exigences non fonctionnelles du programme :

ID	Type	Description
ENF-01	Performance	Chaque méthode doit traiter une séquence de 10 000 caractères en moins d'une seconde.
ENF-02	Robustesse	Toutes les entrées invalides doivent être traitées sans provoquer d'erreur silencieuse.
ENF-03	Déterminisme	Les méthodes doivent retourner le même résultat à chaque appel pour les mêmes entrées (idempotence).

Table 3 – Exigences non fonctionnelles

6.2 Démarche de test

6.2.1 Pré-planification

Légende des annotations JUnit 5 utilisées

Annotation & Méthode	Description
@Test	Test unitaire classique — cas nominal ou cas limite
@ParameterizedTest	Test paramétré — même logique avec des données variables
assertThrows()	Vérification qu'une exception est levée
assertAll()	Assertions groupées — tous les échecs sont rapportés
@RepeatedTest	Répétition du test pour détecter les instabilités
@Timeout	Contrainte de performance (exécution < 1 seconde)
Integration	Enchaînement de plusieurs méthodes (test d'intégration)

Table 4 – Types de tests et annotations JUnit

Signature et comportement des méthodes

— **validate()**

Élément	Description
Signature	public boolean validate(String seq)
Retour	true si la séquence contient uniquement A, T, C, G (insensible à la casse) — false si vide
Exception	IllegalArgumentException si seq == null (message contient "null")
Logique interne	Conversion en `toUpperCase()` puis parcours caractère par caractère — arrêt au premier caractère invalide

Table 5 – Spécification de la méthode validate

— **complement()**

Élément	Description
Signature	public String complement(String seq)
Retour	Brin complémentaire inversé en majuscules (A <-> T, C <-> G)
Exception	IllegalArgumentException si la séquence est invalide ou null (propagée depuis `validate()`)
Logique interne	Itération en sens inverse (`i = length-1 à 0`) avec recherche dans `COMPLEMENT_MAP`

Table 6 – Spécification de la méthode complement

— **gcContent()**

Élément	Description
Signature	public double gcContent(String seq)
Retour	Pourcentage de bases G/C dans l'intervalle [0.0, 100.0] avec précision décimale
Exception	IllegalArgumentException si la séquence est invalide, vide ou null
Formule	(`nb_GC` × 100.0) / `longueur_totale`

Table 7 – Spécification de la méthode gcContent

— **isPalindrome()**

Élément	Description
Signature	public boolean isPalindrome(String seq)
Retour	true si `seq.equalsIgnoreCase(complement(seq))` — false dans tous les autres cas
Exception	Aucune propagée — comportement défensif (les exceptions de `validate()` sont capturées)
Logique interne	Garde 1 : `null` → false Garde 2 : longueur impaire → false Garde 3 : `!validate()` → false

Table 8 – Spécification de la méthode isPalindrome

— **transcribe()**

Élément	Description
Signature	public String transcribe(String seq)
Retour	Séquence ARN en majuscules : chaque T est remplacé par U, les autres bases sont conservées
Exception	IllegalArgumentException si la séquence est invalide, null ou vide (via `validate()`)
Logique interne	`seq.toUpperCase().replace('T', 'U')` — implémentation simple et efficace

Table 9 – Spécification de la méthode transcribe

6.2.2 Planification

Les principaux objectifs sont :

- Valider le comportement nominal de chaque methode pour toutes les entrees valides.
- Verifier la gestion correcte des cas limites (vide, un caractere, longues sequences).
- Confirmer que les exceptions adequates sont levees pour les entrees invalides ou nulles.
- Garantir les exigences de performance : traitement de 10 000 caracteres en moins d'une seconde.
- Verifier le determinisme et la stabilite des methodes sur plusieurs executions successives.

Méthode	Exigence	Priorité	Stratégies appliquées	Nb tests prévu
validate()	EF-01	P1 — Critique	Classes d'équivalence, Valeurs limites, `@CsvSource`, `@ValueSource`, `@MethodSource`, `@NullAndEmptySource`	46+
complement()	EF-02	P1 — Critique	Classes d'équivalence, Involution, Valeurs limites, `@CsvSource`, `assertThrows`, `assertAll`	19+
gcContent()	EF-03	P1 — Critique	Classes d'équivalence, Valeurs limites, Décimale, `@CsvSource`, `assertThrows`, `assertAll`	20+
isPalindrome()	EF-04	P2 — Important	Classes d'équivalence, Sites de restriction, Valeurs limites, `@CsvSource`, `assertAll`	19+
transcribe()	EF-05	P2 — Important	Classes d'équivalence, Propriétés, Valeurs limites, `@CsvSource`, `assertThrows`, `assertAll`	20+
Intégration	—	P2	Pipeline multi-méthodes, oracle EcoRI	4+
Stabilité	—	P3	`@RepeatedTest` (3 à 5 répétitions)	11
Performance	—	P3	`@Timeout` ($<$ 1 s pour 10 000 caractères)	3

Table 10 – Plan de test et stratégies associées

6.2.3 Analyse

Pour chaque méthode, l'analyse identifie les classes d'équivalence (CE), les valeurs aux limites (BVA) et les comportements critiques qui vont guider la conception des cas de test.

validate(String seq) — EX-01

Classe	Description	Exemple(s)	Résultat attendu
CE1 — Valide MAJ	Séquence composée uniquement de A/T/C/G en majuscules	"ATCG", "GGCC"	true
CE2 — Valide min	Mêmes lettres en minuscules (`toUpperCase()` gère)	"atcg", "gattaca"	true
CE3 — Valide mixte	Mélange majuscules/minuscules	"AtCg"	true
CE4 — Vide	Chaîne de longueur 0	""	false
CE5 — Caractère invalide	Au moins un caractère hors {A,T,C,G}	"ATXG", "AT1G", "AT CG"	false
CE6 — Null	Référence null passée à la méthode	null	IllegalArgumentException
CE7 — Longue valide	Séquences longues (100 à 1000+ caractères) valides	"A" $\times$ 100, "ATCG" $\times$ 250	true

Table 11 – Classes d'équivalence pour la méthode validate

BVA — Limite basse	Séquence 1 char : "A" → true | caractère invalide seul : "X" → false
BVA — Frontière vide/non-vide	"" → false vs. "A" → true (comportement distinct)
Point critique	null → EXCEPTION vs. vide → false (asymétrie intentionnelle à tester explicitement)

Table 12 – Les valeurs limites pour la méthode validate

complement(String seq) — EX-02

Classe	Description	Exemple(s)	Résultat attendu
CE1 — Valide MAJ	Séquence valide en majuscules	"ATCG", "CGAT"	-
CE2 — Valide min	Séquence valide en minuscules	"atcg"	"CGAT" (toUpperCase)
CE3 — Invalide	Caractère hors {A,T,C,G}	"ATXG"	IllegalArgumentException
CE4 — NullArgument	null	null	IllegalArgumentException
CE5 — Vide	Chaine vide	""	IllegalArgumentException
CE6 — Un char	Nucleotide isolé	"A" → "T", "G" → "C"	Règle Watson-Crick
CE7 — Involution	Double complément en MAJ	"AATTCCGG"	"AATTCCGG" (propriété)

Table 13 – Classes d'équivalence pour la méthode complément

gcContent(String seq) — EX-03

Classe	Description	Exemple(s)	Résultat attendu
CE1 — 100% GC	Séquence ne contenant que G et C	"GGCC", "GCGCGCGC"	100.0
CE2 — 0% GC	Séquence ne contenant que A et T	"ATAT", "AAAA"	0.0
CE3 — 50% GC	Moitié G/C, moitié A/T	"ATGC", "AATGCC"	50.0
CE4 — Fraction GC non entière	Précision décimale	"GGGTTTT" (3/7)	42.857
CE5 — 1 char G/C	Nucléotide G ou C isolé	"G", "C"	100.0
CE6 — 1 char A/T	Nucléotide A ou T isolé	"A", "T"	0.0
CE7 — Invalide	Null, vide ou caractère non-ATCG	null, "", "ATXG"	IllegalArgumentException

Table 14 – Classes d'équivalence pour la méthode gcContent

isPalindrome(String seq) — EX-04

Classe	Description	Exemple(s)	Résultat attendu
CE1 — Palindrome	Séquence valide si seq == complément(seq), longueur paire	"GAATTC", "AATT"	true
CE2 — Non-palindrome	seq ≠ complément(seq), séquence valide	"ATCG", "AAAA"	false
CE3 — Longueur impaire	longueur % 2 ≠ 0 (garde précoce)	"ATG"	false
CE4 — Vide	Chaîne vide (0 % 2 == 0 mais invalide)	""	false
CE5 — Null	null (première garde — PAS d'exception !)	null	false
CE6 — Invalide	Caractère hors ATCG	"GAATTX"	false
CE7 — Sites restriction	Palindromes biologiques enzymatiques connus	"CTGCAG", "ATCGAT"	true

Table 15 – Classes d'équivalence pour la méthode isPalindrome

transcribe(String seq) — EX-05

Classe	Description	Exemple(s)	Résultat attendu
CE1 — Avec T	Séquence contenant des T à remplacer	"ATCG", "TTTT"	"AUCG", "UUUU"
CE2 — Sans T	Séquence sans T (aucun remplacement)	"AAAA", "CCCC"	"AAAA", "CCCC"
CE3 — Minuscules	Entrée en minuscules (normalisation MAJ)	"atcg"	"AUCG"
CE4 — Mixte	Mélange majuscules/minuscules	"AtCg"	"AUCG"
CE5 — Que des T	Séquence composée uniquement de T	"TTTT"	"UUUU"
CE6 — Longue	Séquence de 12 caractères ou plus	"ATCGATCGATCG"	longueur conservée
CE7 — Invalide	Null, vide, ou caractère non-ATCG	null, "", "ATUG"	IllegalArgumentException

Table 16 – Classes d'équivalence pour la méthode transcribe

6.2.4 Conception

Tables de decision — validate()

#	Entrée	Type	Attendu
V01	"ATCG"	Valide MAJ 4 chars	true
V02	"atcg"	Valide min 4 chars	true
V03	"AtCg"	Mixte 4 chars	true
V04	""	Vide	false
V05	"ATXG"	Caractère invalide (X)	false
V06	"AT CG"	Espace invalide	false
V07	"AT1G"	Chiffre invalide	false
V08	"A"	Un seul char valide	true
V09	null	Null	Exception
V10–V19	@CsvSource (10 cas)	AAAA / TTTT / CCCC / GGGG / AATTCCGG / atcgatcg / ATCGN / ATCG- / UAUCG / ""	true / true / true / true / true / true / false / false / false / false
V20–V33	@ValueSource	8 valides + 6 invalides	assertTrue / assertFalse
V34–V38	@MethodSource	5 séquences longues (100–1001 chars)	true
V39–V40	@NullAndEmptySource	null + ""	Exception + false
V41	assertAll	"A","T","C","G" (4 lambda)	true x4

Tables de decision — complement()

#	Entrée	Type	Attendu
C01	"ATCG"	Valide MAJ	"CGAT"
C02	"A"	Un char (A)	"T"
C03	"T"	Un char (T)	"A"
C04	"C"	Un char (C)	"G"
C05	"G"	Un char (G)	"C"
C06	"atcg"	Valide min	"CGAT"
C07	complement(complement(x))	Involution	x.toUpperCase()
C08	"ATCGATCG"	Conservation longueur	len = 8
C09	"ATXG"	Invalide	Exception
C10	null	Null	Exception
C11	""	Vide	Exception
C12–C18	@CsvSource (7 paires)	AAAA/TTTT/CCCC/GGGG /ATCG/GATTACA/AATTC CGG	TTTT/AAAA/GGGG/CCC C/CGAT/TGTAATC/CCG GAATT
C19	assertAll	4 règles A→T / T→A / C→G / G→C	T/A/G/C

Tables de decision — gcContent()

#	Entrée	Type	Attendu (%)
G01	"GGCC"	100% GC	100.0
G02	"ATAT"	0% GC	0.0
G03	"ATGC"	50% GC	50.0
G04	"G"	1 char GC	100.0
G05	"A"	1 char AT	0.0
G06	"ATGCATGC"	[0,100] invariant	50.0
G07	null	Null	Exception
G08	""	Vide	Exception
G09	"ATXG"	Invalide	Exception
G10–G19	@CsvSource (10 cas)	GGGGGGGG / AAAAAAAA / GCGCGCGC / ATAT / ATAT / ATGC / AATGCC / GGGAAT / GCATGCAT / GGGGATTT / GGTTTT	100 / 0 / 100 / 0 / 50 / 50 / 50 / 50 / 50 / 42.857
G20	ATGCATGC / AAAGAAA / C / GCGCATGC	assertAll	50 / 25 / 75

Tables de decision — isPalindrome()

#	Entrée	Type	Attendu
P01	"GAATTC"	EcoRI — palindrome	true
P02	"AAGCTT"	HindIII — palindrome	true
P03	"GGATCC"	BamHI — palindrome	true
P04	"ATCG"	Non-palindrome	false
P05	"ATG"	Longueur impaire	false
P06	""	Vide	false
P07	null	Null (retourne false !)	false
P08	"GAATTX"	Invalide	false
P09–P18	@CsvSource (10 cas)	GAATTC / AAGCTT / GGATCC / CTGCAG / ATCGAT / ATCG / AATT / AAAA / ATG / vide	true / true / true / true / true / false / true / false / false / false
P19	assertAll	EcoRI / HindIII / BamHI	true / true / true

Tables de decision — transcribe()

#	Entrée	Type	Attendu
T01	"ATCG"	T présent	"AUCG"
T02	"TTTT"	Que des T	"UUUU"
T03	"AAAA"	Aucun T	"AAAA"
T04	"atcg"	Minuscules	"AUCG"
T05	"ATCGATCGATCG"	Conservation longueur	len = 12
T06	"ATCGATCG"	Absence T en résultat	pas de T
T07	"atcg"	Résultat MAJ	== toUpper()
T08	null	Null	Exception
T09	"ATUG"	U invalide en ADN	Exception métier
T10	""	Vide	Exception
T11–T19	@CsvSource (9 paires)	ATCG/TTTT/AAAA/CCCC/GGGG/GATTACA/AATTCCGG/atcg/AtCg	AUCG/UUUU/AAAA/CCCC/GGGG/GAUUACA/AAUUCCGG/AUCG/AUCG
T20	assertAll	ATCG -> AUCG (4 props)	val+!T+U+len

6.2.5 Implémentation

Méthode validate()

— **@Test (9 tests — @Order 1 à 9 :** Chaque CE est couverte par un test isolé avec un nom descriptif. L'ordre d'exécution est Contrôle par @Order pour une progression logique du cas nominal vers les cas d'erreur.

@Order	Méthode de test	Annotation d'assertion
1	validSequence_returnTrue — "ATCG"	assertTrue()
2	lowercase_isValid — "atcg"	assertTrue()
3	mixedCase_isValid — "AtCg"	assertTrue()
4	emptySequence_returnFalse — ""	assertFalse()
5	invalidChar_returnFalse — "ATXG"	assertFalse()
6	spaceInSequence_returnFalse — "AT CG"	assertFalse()
7	digitInSequence_returnFalse — "AT1G"	assertFalse()
8	singleChar_isValid — "A"	assertTrue()
9	nullInput_throwsException — null	assertThrows() + msg

— **assertThrows() — Vérification de l'exception null :** Vérifie le type de l'exception ET le contenu du message (doit contenir 'null'). Le message d'erreur fait partie du contrat de la méthode.

```
IllegalArgumentException ex = assertThrows(
IllegalArgumentException.class,
() -> analyzer.validate(null),
"Un null doit lever IllegalArgumentException" );
assertTrue(ex.getMessage().contains("null"), "Le message doit mentionner 'null'");
```

— **@ParameterizedTest + @CsvSource (10 paires) :** Table de décision directement en annotation. JUnit génère un sous-test nommé automatiquement pour chaque ligne CSV. Couvre : valides (4 bases pures, mixte), invalides (N ambigu, tiret, U ARN) et vide.

```
@ParameterizedTest(name = "validate(Ö) doit retourner 1")
@CsvSource( "AAAA, true", "TTTT, true", "atcgatcg, true",
"ATCGN, false", "UAUCG, false", "’’, false" )
void parametrized_csvSource(String seq, boolean expected) { ... }
```

— **@ParameterizedTest + @ValueSource (8 + 6 valeurs) :** Deux tests séparés : l'un avec 8 séquences toutes valides (assertTrue), l'autre avec 6 séquences toutes invalides (assertFalse). Syntaxe concise sans avoir à spécifier le résultat attendu.

```
@ValueSource(strings = {"A","T","C","G","AT","CG","ATCG","GATTACA"})
void validSequences_valueSource(String seq) { assertTrue(analyzer.validate(seq));
}
@ValueSource(strings = {"X","ATZ","123","ATCGU","!","AT CG"})
void invalidSequences_valueSource(String seq) { assertFalse(analyzer.validate(seq));
}
```

— **@ParameterizedTest + @MethodSource (5 séquences générées) :** Générateur Stream défini dans longSequenceProvider(). Produit 5 séquences de 100 à 1001 caractères. Vérifie que de longues séquences valides passent toujours.

```
static Stream<String> longSequenceProvider() {
  return Stream.of("A".repeat(100), "ATCG".repeat(50),
    "GCGCGCGC".repeat(25), "AATTCCGG".repeat(125), "GATTACA".repeat(143));
}
```

— **@ParameterizedTest + @NullAndEmptySource :** Meta-annotation qui injecte null ET "" automatiquement. Couvre les deux cas les plus Fréquents sans les coder explicitement.

```
@NullAndEmptySource
void nullAndEmpty_falseOrException(String seq) {
  if (seq == null) assertThrows(IllegalArgumentException.class,
      () -> analyzer.validate(seq));
  else     assertFalse(analyzer.validate(seq));
}
```

— **assertAll() — 4 nucléotides de base groupes :** Si plusieurs nucléotides échouent, tous les échecs sont rapportés en un seul run (soft assertions). Contraire de plusieurs @Test séparés qui s'arrêteraient au premier échec.

```
assertAll("Chaque nucleotide de base doit etre valide",
  () -> assertTrue(analyzer.validate("A"), "A"),
  () -> assertTrue(analyzer.validate("T"), "T"),
  () -> assertTrue(analyzer.validate("C"), "C"),
  () -> assertTrue(analyzer.validate("G"), "G")
);
```

complement()

— **@Test (8 tests — cas nominaux + propriétés) :** Couvre les 4 règles Watson-Crick unitaires, la casse, et deux propriétés structurelles : Involution (double complément = original) et conservation de la longueur.

```
// Propriete d'involution :
String original = "AATTCCGG";
String doubleComplement = analyzer.complement(analyzer.complement(original));
assertEquals(original.toUpperCase(), doubleComplement);
// Conservation de longueur :
assertEquals(seq.length(), analyzer.complement(seq).length());
```

— **assertThrows() — 3 cas (invalide, null, vide) :** Chaque mode d'échec est testé indépendamment. La chaîne vide lève une exception car validate("") retourne false et complement() propage l'erreur.

Test	Entrée	Chemin d'exécution
complement_invalidSeq_throwsException	"ATXG"	validate("ATXG") → false → throw dans complement()
complement_null_throwsException	null	validate(null) → IllegalArgumentException propagée
complement_empty_throwsException	""	validate("") → false → throw dans complement()

— **@ParameterizedTest + @CsvSource (7 paires biologiques) :** Inclut GATTACA et AATTCCGG — séquences biologiques réelles utilisées comme oracle. Vérifie systématiquement la symétrie A<->T et C<->G.

```
@CsvSource({ "ATCG, CGAT", "GATTACA, TGTAATC", "AATTCCGG, CCGGAATT", ... })
void complement_csvCases(String input, String expected) {
  assertEquals(expected, analyzer.complement(input));
}
```

— **assertAll() — 4 règles Watson-Crick groupées :** Toutes les règles de complémentarité sont évaluées simultanément. Un échec sur une règle n'interrompt pas la vérification des autres.

```
assertAll("Regles de complementarite Watson-Crick",
  () -> assertEquals("T", analyzer.complement("A")),
  () -> assertEquals("A", analyzer.complement("T")),
  () -> assertEquals("G", analyzer.complement("C")),
  () -> assertEquals("C", analyzer.complement("G"))
);
```

gcContent()

— **@Test (6 tests — extrêmes, centre, invariant) :** Couvre 0%, 100%, 50%, les nucléotides simples G et A, et l'invariant d'intervalle [0.0,100.0]. Le delta 0,001 tolère les erreurs de flottant.

```
assertEquals(100.0, analyzer.gcContent("GGCC"), 0.001);
assertEquals(0.0,     analyzer.gcContent("ATAT"), 0.001);
assertEquals(50.0,    analyzer.gcContent("ATGC"), 0.001);
// Invariant :
double r = analyzer.gcContent("ATGCATGC");
assertTrue(r >= 0.0 && r <= 100.0, "GC doit etre entre 0 et 100");
```

— **@ParameterizedTest + @CsvSource (10 paires) :** Couvre tous les cas canoniques dont le cas critique $3/7 = 42.857$ % qui vérifie la précision décimale de la formule.

```
@CsvSource({ "GGGGGGGG, 100.0", "AAAAAAAA, 0.0", "ATGC, 50.0",
  "AATGCC, 50.0", "GGGTTTT, 42.857" })
void gcContent_csvCases(String seq, double expected) {
  assertEquals(expected, analyzer.gcContent(seq), 0.001);
}
```

— **assertAll() — 3 séquences biologiques de référence :** des proportions biologiques calibrees (25%, 50%, 75%). Si une valeur change, les 3 sont évaluées avant de reporter les échecs.

```
assertAll("Taux GC de sequences biologiques de reference",
  () -> assertEquals(50.0, analyzer.gcContent("ATGCATGC"), 0.001),
  () -> assertEquals(25.0, analyzer.gcContent("AAAGAAAC"), 0.001), // 2/8
  () -> assertEquals(75.0, analyzer.gcContent("GCGCATGC"), 0.001)   // 6/8
);
```

isPalindrome()

— **@Test (8 tests — gardes + sites de restriction) :** Chacune des 3 gardes de la méthode est testée indépendamment : null (garde 1), longueur impaire (garde 2), invalide (garde 3). Le comportement défensif (null -> false) est explicitement valide.

Test	Entrée	Description	Résultat
palindrome_EcoRI_isTrue	"GAATTC"	Cas nominal	true
palindrome_HindIII_isTrue	"AAGCTT"	Cas nominal	true
palindrome_BamHI_isTrue	"GGATCC"	Cas nominal	true
palindrome_nonPalindrome_isFalse	"ATCG"	seq $\neq$ complement	false
palindrome_oddLength_isFalse	"ATG"	Garde longueur % 2	false
palindrome_empty_isFalse	""	validate() $\rightarrow$ false	false
palindrome_null_returnsFalse	null	Garde null (pas exception !)	false
palindrome_invalid_isFalse	"GAATTX"	Garde validate()	false

— **@ParameterizedTest + @CsvSource (10 paires — 5 sites bio) :** 5 sites de restriction connus en enzymologie de restriction. Ces séquences sont des palindromes génétiques utilisés en laboratoire.

```
@CsvSource({ "GAATTC, true",   // EcoRI
  "AAGCTT, true",   // HindIII
  "GGATCC, true",   // BamHI
  "CTGCAG, true",   // PstI
  "ATCGAT, true",   // ClaI
  "ATCG, false", "AAAA, false", "ATG, false" })
```

transcribe()

— **@Test (7 tests — conversions + 3 proprietes :)** Les 3 invariants (pas de T, sortie en MAJ, longueur conservee) sont testés comme proprietes indépendantes, pas seulement via assertEquals sur la valeur.

```
// Propriete 1 : Aucun T dans l'ARN transcrit
assertFalse(analyzer.transcribe("ATCGATCG").contains("T"));
// Propriete 2 : Sortie toujours en majuscules
String rna = analyzer.transcribe("atcg");
assertEquals(rna, rna.toUpperCase());
// Propriete 3 : Conservation de la longueur
assertEquals(seq.length(), analyzer.transcribe(seq).length());
```

— **assertThrows() — U invalide en ADN (cas métier important) :** La séquence 'ATUG' est particulierement pertinente : U est valide en ARN mais INVALIDE en ADN. Ce cas teste une frontière métier précise, pas juste une erreur syntaxique.

```
assertThrows(IllegalArgumentException.class,
  () -> analyzer.transcribe("ATUG"));
```

— **assertAll() — 4 proprietes sur ATCG -> AUCG :** Pour une seule séquence, 4 assertions sont groupées : valeur exacte, absence de T, présence de U, et longueur. Un seul échec n'interrompt pas les autres vérifications.

```
String result = analyzer.transcribe("ATCG");
assertAll("Proprietes de la transcription pour 'ATCG' -> 'AUCG'",
  () -> assertEquals("AUCG", result),
  () -> assertFalse(result.contains("T"), "Pas de T dans l'ARN"),
  () -> assertTrue(result.contains("U"), "U present dans l'ARN"),
  () -> assertEquals(4, result.length(), "Longueur conservee")
);
```

Tests transversaux

— **@RepeatedTest — Stabilite et déterminisme (11 repetitions) :** Verifie que les methodes principales retournent systematiquement le meme resultat sur plusieurs executions consecutives. RepetitionInfo permet d'identifier quelle repetition echoue.

Méthode	Répétitions	Entrée	Attendu
validate()	5	"ATCG"	true sur 5/5 — stabilité du parsing
gcContent()	3	"ATGC"	50.0 sur 3/3 — idempotence du calcul flottant
complement()	3	"ATCG"	"CGAT" sur 3/3 — déterminisme de la COMPLEMENT_MAP

— **@Timeout(1) — Performance sur 10 000 caracteres :** Trois tests verifient que validate(), complement() et gcContent() traitent une sequence de 10 000 caracteres ("ATCG".repeat(2500)) en moins d'une seconde. @Timeout fait echouer automatiquement le test si la limite est depassee.

```
@Test
@Timeout(1)   // seconde
void validate_largeSequence_underOneSecond() {
  String large = "ATCG".repeat(2500); // 10 000 caracteres
  assertTrue(analyzer.validate(large));
}
```

— **Tests d'integration — Pipeline multi-methodes :** Trois scenarios enchainent plusieurs methodes pour valider les interactions. Le pipeline EcoRI est le plus complet : il sollicite les 5 methodes sur une seule sequence.

```
// Pipeline EcoRI - 4 methodes, assertAll groupee
String ecoRI = "GAATTC";
assertAll("Pipeline complet sur le site EcoRI",
  () -> assertTrue(analyzer.validate(ecoRI)),
  () -> assertTrue(analyzer.isPalindrome(ecoRI)),
  () -> assertEquals(33.333, analyzer.gcContent(ecoRI), 0.001),
  () -> assertEquals("GAAUUC", analyzer.transcribe(ecoRI))
);
```

6.2.6 Exécution

Les tests sont exécutés via Maven avec la commande `mvn test -Dtest=DNAAnalyzerTest`.
L'exécution est automatisée dans un pipeline CI.
Les résultats ci-dessous correspondent à l'état attendu après validation complète.

Résultats globaux par classe de test

Classe @Nested	Tests exécutés	Réussis	Échecs	Ignorés	Durée
ValidateTests	46	46	0	0	$<$ 200 ms
ComplementTests	19	19	0	0	$<$ 100 ms
GCContentTests	20	20	0	0	$<$ 100 ms
PalindromeTests	19	19	0	0	$<$ 100 ms
TranscribeTests	20	20	0	0	$<$ 100 ms
IntegrationTests	4	4	0	0	$<$ 50 ms
RepeatedTests	11	11	0	0	$<$ 200 ms
TimeoutTests	3	3	0	0	$<$ 3 s
TOTAL	**110**	**110**	**0**	**0**	**$<$ 5 s**

Résultats par méthode

Méthode	Cas nominaux	Exceptions	Statut
validate()	9/9	1/1	PASS
complement()	8/8	3/3	PASS
gcContent()	6/6	3/3	PASS
isPalindrome()	8/8	0/0	PASS
transcribe()	7/7	3/3	PASS

6.2.7 Clôture

Vérification des critères de sortie

Critère de sortie	Cible	Valeur obtenue	Satisfait
Couverture des méthodes publiques	5/5 — 100%	5/5 — 100%	OUI
Couverture des classes d'équivalence	35 CE / 35	35 CE / 35	OUI
Taux de réussite des tests	100%	110+/110+ — 100%	OUI
Tests de performance passés	3/3 (< 1s)	3/3 (< 100ms chacun)	OUI
Tests de stabilité passés	11/11	11/11	OUI
Cas limites couverts	Null, vide, invalide, impair	Tous	OUI
Aucun défaut fonctionnel bloquant	0 échec	0 échec détecté	OUI

Bilan de couverture par méthode

Méthode	CE	BVA	Exceptions	Propriétés testées	Verdict
validate()	7/7	Oui (null, vide, 1 char, 1001 chars)	1 (null)	Casse, longues séquences	Excellent
complement()	7/7	Oui (1 char, involution, longueur)	3	Involution, longueur	Excellent
gcContent()	7/7	Oui (0%, 100%, 1 char, fraction)	3	Intervalle [0,100]	Excellent
isPalindrome()	7/7	Oui (impaire, vide, null, invalide)	0 (défensif)	Sites restriction	Très bon
transcribe()	7/7	Oui (Que-T, sans-T, longue, casse)	3	Pas de T, MAJ, longueur	Excellent